

7.1 Explain the JavaFX application structure (Application, Stage, Scene, Grid Pane / VBox). Role of start() ?

JavaFX - is a java framework for creating graphical user interfaces.

A JavaFX application normally extends the Application class.

```
public class Main extends Application {
    @Override
    public void start(Stage stage) {
        // GUI code
    }
}
```

Important components -

- The main class of a JavaFX application extends Application .
- Stage - The stage represents the main window.
- Scene - Contains all GUI components displayed inside the stage.
- Grid Pane - arranges controls in rows and columns
- VBox - arranges components vertically

Role of start() - the start() method is the main entry point for building the JavaFX user interface.)

Example -

@Override

```
public void start (Stage stage) {  
    // create GUI  
}
```

7.2 Design the GridPane layout: Loan Amount, Annual Rate, Number of Years inputs + "Calculate" button + result labels.

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class LoanCalculator extends Application {
```

@Override

```
public void start(Stage stage) {
    GridPane grid = new GridPane();
    grid.setPadding(new Insets(10));
    grid.setHgap(10);
    grid.setVgap(10);
```

```
Label loanLabel = new Label("Loan Amount: ");
```

```
TextField loanField = new TextField();
```

```
Label rateLabel = new Label("Annual Rate (%): ");
```

```
TextField rateField = new TextField();
```

```
Label yearsLabel = new Label("Years: ");
TextField yearsField = new TextField();
```

```

Button calculateButton = new Button("Calculate");
Label monthlyLabel = new Label("Monthly Payment:");
Label totalLabel = new Label("Difference:");

```

```

grid.add(loanLabel, 0, 0);
grid.add(loanField, 1, 0);
grid.add(rateLabel, 0, 1);
grid.add(rateField, 1, 1);
grid.add(yearsLabel, 0, 2);
grid.add(yearField, 1, 2);
grid.add(calculateButton, 1, 3);
grid.add(monthlyLabel, 0, 4);
grid.add(totalLabel, 0, 5);
grid.add(differenceLabel, 0, 6);

```

```

Scene scene = new Scene(grid, 400, 300);

```

```

stage.setTitle("House Loan Calculator");

```

```

stage.setScene(scene);

```

```

stage.show();
}

```

```

public static void main(String[] args) {

```

```

    launch(args);
}
}

```

Layout:

Loan Amount: []

Annual Rate(%): []

Number of Years: []

[Calculate]

Monthly Payment:

Total Payment :

Difference :

7.3 Event handler computing Monthly Installment (amortization formula), Total Payment, and Difference (Total Payment - Loan Amount).

Formula -

Let : P = Loan Amount

R = Monthly interest rate

n = Total number of monthly payments

Monthly rate : $r = \text{annual Rate} / 100 / 12$

Number of payments : $n = \text{years} \times 12$

Monthly installment : $M = P \times r \times (1+r)^n / ((1+r)^n - 1)$

Total payment : $\text{Total} = M \times n$

Difference : $\text{Difference} = \text{Total Payment} - \text{Loan Amount}$

Event handler -

```

calculateButton.setOnAction(e -> {
    double loan = Double.parseDouble(
        loanField.getText()
    );
    double annualRate = Double.parseDouble(
        rateField.getText()
    );
    int years = Integer.parseInt(
        yearsField.getText()
    );
    double monthlyRate = annualRate / 100 / 12 ;
    int months = years * 12 ;
    double monthlyPayment ;
    if (monthlyRate == 0) {
        monthlyPayment = loan / months ;
    } else {
        monthlyPayment = loan * monthlyRate *
            Math.pow (1 + monthlyRate, months)
            / (Math.pow (1 + monthlyRate, months) - 1) ;
    }
}

```

```
double totalPayment = monthlyPayment * months;  
double difference = total Payment - loan;  
monthlyLabel.setText(  
    "Monthly Payment: " + monthlyPayment  
);  
totalLabel.setText(  
    "Total Payment: " + total payment);  
differenceLabel.setText("Difference: " +  
    difference);  
} );
```

8.1 Compare Socket Programming vs Java RMI . When to prefer each ?

Socket programming and Java RMI , both are techniques for communication between programs running on different machine.

Socket	Java RMI
Lower-level communication	Higher level communication
User IP address and port	User remote objects
Developer handles input/output streams	Java handles much of communication
Can be used between different languages	Mainly java-to-java
More control	Easier object oriented approach
Good for custom chat/network protocols	Good for distributed Java applications

Socket is preferred when -

- Building chat application
- Developing custom client-server protocols.
- Need more control over communication.

RMI is preferred when -

- Both client-server are Java
- We want to call methods of a remote Java object.
- Object-oriented distributed programming is required.

8.2 Server code using ServerSocket: accept connection, read via BufferedReader, reply via PrintWriter.

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main (String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println ("Server started ...");
            System.out.println ("Waiting for client ...");
            Socket socket = serverSocket.accept ();
            BufferedReader in = new BufferedReader (
                new InputStreamReader (
                    socket.getInputStream ()
                )
            );
            PrintWriter out = new PrintWriter (
                socket.getOutputStream (),
                true
            );
            String message = in.readLine ();
            System.out.println ("Client: " + message );
            out.println ("Message received by server.");
        }
    }
}
```

```
socket.close();  
serverSocket.close();  
} catch (IOException e) {  
    System.out.println (e.getMessage());  
} } }
```

Main steps -

```
graph TD  
    A[create ServerSocket] --> B[accept() client]  
    B --> C[Read message]  
    C --> D[process message]  
    D --> E[Send Reply]  
    E --> F[Close connection.]
```

8.3 Client code using Socket: connect to server, send typed message, print server reply.

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main (String[] args){
        try {
            Socket socket = new Socket ("localhost", 5000);
            BufferedReader in = new BufferedReader (
                new InputStreamReader (
                    socket.getInputStream()
                )
            );
            PrintWriter out = new PrintWriter (socket.getOutputStream(),
                true
            );
            Scanner input = new Scanner (System.in);
            System.out.print ("Enter message: ");
            String message = input.nextLine();
            out.println (message);
            String reply = in.readLine();
```

```
System.out.println ("Server : " + reply);  
Socket.close ();  
  
} catch (IOException e) {  
    System.out.println (e.getMessage());  
}  
}  
}.
```

Execution-

Firstly run : Server

Then run : Client

Example : Client : Hello Server

Server: Message received by server.

9.4 Client code using Socket : connect to server, send typed message, print server reply.

Step 1 - Create Database

```
CREAT DATABASE student_db;
```

Step 2 - Select Database

```
USE student_db;
```

Step 3 - Create Table

```
CREAT TABLE Students (  
    id INT PRIMARY KEY,  
    name VARCHAR (100),  
    cgpa DOUBLE  
);
```

Step 4 - Add JDBC Driver

Add MySQL Connector/J to the project.

Step 5 - Connection String

```
String url = "jdbc:mysql://localhost:3306/student_db";
```

```
String username = "root";
```

```
String password = "1234";
```

Step 6 - Servlet Mapping

Using annotation :

```
@WebServlet("/student")
```

The request :

/student

will be handled by the servlet.

Basic flow -

HTML/JSP Form



Servlet



JDBC



MySQL



Servlet/JSP Response

9.2 Servlet doPost(): read ID/Name/CGPA from form, insert via PreparedStatement, with exception handling.

```
import java.io.* ;
import java.sql.* ;
import jakarta.servlet.* ;
import jakarta.servlet.http.* ;
import jakarta.servlet.annotation.* ;
```

@WebServlet ("/student").

```
public class StudentServlet extends HttpServlet {
    protected void doPost (
```

```
        HttpServletRequest request,
```

```
        HttpServletResponse response)
```

```
    throws ServletException, IOException {
```

```
        int id = Integer.parseInt (
            request.getParameter ("id")
        );
```

```
        String url = "jdbc:mysql://localhost:3306/student_db";
```

```
        String username = "root" ;
```

```
        String password = "1234" ;
```

```
        String sql =
```

```
            "INSERT INTO students(id, name, cgpa)" +
```

```
            "VALUES (?, ?, ?)";
```

```

try {
    Connection con = DriverManager.getConnection(
        url, username, password
    );
    PreparedStatement ps = con.prepareStatement(sql);

    ps.setInt(1, id);
    ps.setString(2, name);
    ps.setDouble(3, gpa);

    ps.executeUpdate();
    response.getWriter().println(
        "student inserted successfully."
    );
    ps.close();
    con.close();
} catch (SQLException e) {
    response.getWriter().println(
        "Database Error: " + e.getMessage()
    );
}
}
}

```

important: Prepared statement is preferred because it -

- Uses permanent
- Makes SQL easier to handle
- Helps protect against SQL injection

9.3 JSP page: connect to student-db, SELECT from Students, display as HTML table.

```
<%@ page import = "java.sql.*" %>
```

```
<html>
```

```
<head>
```

```
<title> Student List </title>
```

```
</head>
```

```
<body>
```

```
<h2> Student Records </h2>
```

```
<table border = "1" >
```

```
<tr>
```

```
<th> ID </th>
```

```
<th> Name </th>
```

```
<th> CGPA </th>
```

```
</tr>
```

```
<%
```

```
String url = "jdbc:mysql://localhost:3306/student-db";
```

```
String username = "root";
```

```
String password = "1234";
```

```
String sql = "SELECT id, name, cgpa FROM Students";
```

```
try {
```

```
Connection con = DriverManager.getConnection(
```

```
url, username, password
```

```

);
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    %.>
}
rs.close();
ps.close();
con.close();
} catch (SQLException e) {
    out.println (
        "Error: " + e.getMessage()
    );
}
%.>
</table>
</body>
</html>

```

Output concept :

ID	Name	CGPA
1	Rahim	3.50
2	Ratul	3.75
3	Raha	3.90